- Compute Match — Environments, Infrastructure, Delivery and CI
  - Environments
  - Google Cloud infrastructure (Terraform)
  - Helm
  - Application release train
  - GitLab CI
  - Git workflow (CONTRIBUTING.md)

Compute Match — Environments, Infrastructure, Delivery and CI

Environments

Environment	How applications are delivered
Dev	Continuously from protected `main`
QA	Manual promotion of a qualified Dev snapshot
Staging (STG)	Manual promotion of the immutable QA package (`STG_RELEASE_SNAPSHOT_ID`)
Production	Not enabled from the pipeline until authorized

Infrastructure for each environment is applied by a separate Terraform pipeline. Environment values live in `infrastructure/environments/<env>/` .

Google Cloud infrastructure (Terraform)

Stacks in `infrastructure/terraform/stacks/` , applied in order: `project-bootstrap` → `environment-bootstrap` → `computematch` → `platform` → `keycloak` .

- **computematch** stack: VPC, **GKE Autopilot** private cluster, **Cloud SQL PostgreSQL**, **Memorystore Redis**, Secret Manager metadata, Workload Identity, edge and Cloud Armor.
- **platform** stack: namespaces, Kubernetes service accounts, RBAC, NetworkPolicy baselines, Gateway resources.
- **keycloak** stack: declarative realm and identity-provider configuration.

Terraform owns cloud resources; Helm owns workloads. Runtime secrets live only in Google Secret Manager.

Helm

Charts in `infrastructure/helm/`: workloads `backend`, `frontend`, `workflow-worker`, `keycloak`, `superset`; platform `db-bootstrap`, `otel-collector`; library `common`. Deployment order: db-bootstrap → otel-collector → keycloak → backend → worker → frontend → superset. Workloads use SecretSync and Cloud SQL Auth Proxy sidecars; image digests are pinned in environment values.

Application release train

“Build once, promote digests.” Components: backend, frontend, worker, superset, keycloak.

1. Merge to `main` publishes images tagged by commit SHA and mirrors them to Artifact Registry.
2. Dev auto-deploys changed components, runs smoke and functional checks, and records a snapshot.
3. QA promotes the exact digests and creates the package `computematch-release/<snapshot-id>` (manifest plus checksum).
4. Staging consumes that package without rebuilding.
5. Production promotion is not yet enabled.

GitLab CI

Stages: test → security → build → publish → sync → plan → infrastructure → platform → deploy → verify.

- Merge requests: path-scoped tests, security scans (secret scan, IaC SAST), image builds without publishing, read-only Terraform plans.
- `main`: publish images, mirror, Dev release. Infrastructure applies are manual on protected `main` and independent of application releases.

Git workflow (CONTRIBUTING.md)

- Branch: `NCPMS-<number>/<kebab-case-outcome>`, for example `NCPMS-565/repository-foundation`. One Jira issue per branch; branch from updated `main`; no long-lived develop/staging branches.
- Commit: `<type>(<scope>): NCPMS-<number> <imperative description>`, for example `fix(backend): NCPMS-855 widen hosting provider name columns`. Types: feat, fix, docs, test, refactor, perf, build, ci, chore, revert. Add `!` for breaking changes.
- Merge request title: `NCPMS-<number>: <Capitalized outcome>`. Description sections: Objective, Changes, Validation, Risk and rollback, Tracking.
- Labels: one `type::`, one `risk::` (low, medium, high), one or more `component:`; optional `review:security`, `review:database`, `review:platform`, `breaking-change`.
- No direct pushes or force pushes to `main`; pipelines must pass and discussions be resolved; fast-forward merges; no whole-MR squash.
- Install hooks with `pre-commit install` and run `pre-commit run --all-files`.